

Adaptive Spatial Goodness Encoding: Advancing and Scaling Forward-Forward Learning Without Backpropagation

Qingchun Gong¹, Robert Bogdan Staszewski¹, Kai Xu²

¹University College Dublin

²King's College London

qingchun.gong@ucdconnect.ie, robert.staszewski@ucd.ie, kai.xu@kcl.ac.uk

Abstract

The Forward-Forward (FF) algorithm offers a promising alternative to backpropagation (BP). Despite advancements in recent FF-based extensions, which have enhanced the original algorithm and adapted it to convolutional neural networks (CNNs), they often suffer from limited representational capacity and poor scalability to large-scale datasets, primarily due to exploding channel dimensionality. In this work, we propose adaptive spatial goodness encoding (ASGE), a new FF-based training framework tailored for CNNs. ASGE leverages feature maps to compute spatially-aware goodness representations at each layer, enabling layer-wise supervision. Crucially, this approach decouples classification complexity from channel dimensionality, thereby addressing the issue of channel explosion and achieving competitive performance compared to other BP-free methods. ASGE outperforms all other FF-based approaches across multiple benchmarks, delivering test accuracies of 99.65% on MNIST, 93.41% on FashionMNIST, 90.62% on CIFAR-10, and 65.42% on CIFAR-100. Moreover, we present the first successful application of FF-based training to ImageNet, with Top-1 and Top-5 accuracies of 26.21% and 47.49%. By entirely eliminating BP and significantly narrowing the performance gap with BP-trained models, the ASGE framework establishes a viable foundation toward scalable BP-free CNN training.

Introduction

Artificial Neural Networks (ANNs) have become a cornerstone of modern machine learning, largely owing to the effectiveness of the backpropagation (BP) algorithm (Rumelhart, Hinton, and Williams 1986). BP has driven the success of ANNs across a diverse range of applications (Goodfellow et al. 2014; He et al. 2015; LeCun, Bengio, and Hinton 2015; Kingma and Ba 2017; Vaswani et al. 2023) by enabling efficient training through gradient-based optimization. The algorithm proceeds in two phases: a forward pass computes the network’s predictions and their associated error relative to the ground truth, followed by a backward pass that propagates this error through the network. This yields gradients for each layer’s parameters (weights and biases), which are then used by optimization methods such as stochastic gradient descent (SGD) (Bottou 2010) to iteratively minimize

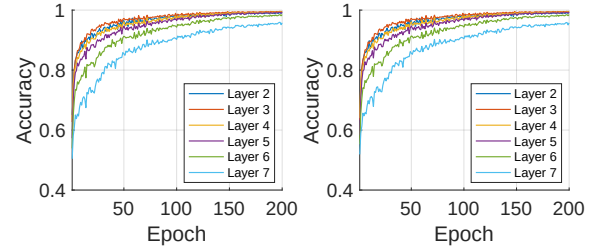


Figure 1: Layer-wise training accuracy (layers 2-7) of VGG8 trained with FF on CIFAR-10. First and final layers excluded. Left: positive samples. Right: negative samples.

the prediction error. However, BP suffers from inherent limitations. It requires symmetric weight matrices for the forward and backward passes to compute parameter gradients via the chain rule (Grossberg 1987). Additionally, it necessitates the storage of intermediate activations until parameter updates occur (Czarnecki et al. 2017). These constraints incur substantial computational and memory overhead, while simultaneously restricting the degree of parallelization, ultimately rendering BP inefficient and ill-suited for deployment on resource-constrained hardware.

In response, numerous backpropagation-free (BP-free) strategies have emerged to address these limitations, including Target Propagation (TP) (Bengio 2014), Feedback Alignment (FA) (Lillicrap et al. 2016), Direct Feedback Alignment (DFA) (Nøkland 2016), Deep Random Target Projection (DRTP) (Frenkel, Lefebvre, and Bol 2021), and Recursive Local Representation Alignment (rLRA) (Ororbia et al. 2023), alongside other biologically inspired or local learning methods (Hebb 2002; Moraitis et al. 2022; Dellaferrera and Kreiman 2023). Although these approaches alleviate some of the BP limitations, they continue to face challenges in balancing the trade-offs between prediction accuracy, memory efficiency and computational cost.

A recent promising alternative is the Forward-Forward algorithm (FF) (Hinton 2022), which employs a layer-wise training strategy through dual forward passes: one with positive samples and the other with negative samples. Each layer independently updates its parameters to maximize a “goodness” metric for positive inputs while minimizing it for negative ones. By eliminating the need for backward

passes, FF significantly reduces memory overhead and facilitates pipelined parallelism. However, Figure 1 illustrates a critical limitation of FF when applied to convolutional neural networks (CNNs): prediction accuracy tends to deteriorate progressively with network depth. This indicates a weakening capacity for feature abstraction in deeper layers, thereby constraining the algorithm’s effectiveness in deep architectures and on complex tasks. Recent enhancements, such as Channel-wise Competitive (CwC) learning (Papachristodoulou et al. 2024) and DeeperForward (Sun et al. 2025), attempt to mitigate this issue. While partially effective, they introduce an undesirable coupling between channel dimensionality and classification complexity. Specifically, they require layer-wise channels C_l to scale linearly with classes N , i.e., $C_l \in \Theta(N)$. This scaling becomes impractical for large-scale datasets like ImageNet (Russakovsky et al. 2015) due to the dramatic increase in channel dimensionality.

To address both the representational degradation and scalability bottleneck in existing FF-based methods, we propose an enhanced FF-based training framework that simultaneously improves representational capacity and scalability to large-scale datasets. Our key contributions are as follows:

- **Adaptive Spatial Goodness Encoding (ASGE) Framework:** A novel BP-free training method that decouples channel dimensionality from classification complexity, enforcing constant channel scaling ($C_l \in \Theta(1)$). This enables efficient and scalable optimization on large-category datasets.
- **ImageNet Breakthrough:** Comprehensive experiments across various image classification tasks, and first successful scaling of FF-based methods to ImageNet.

Background

BP (Rumelhart, Hinton, and Williams 1986) enables end-to-end optimization by propagating errors from the output layer back through the network. However, it suffers from several inherent limitations. First, it requires weight symmetry between the forward and backward passes (Grossberg 1987). Second, BP introduces the update locking problem (Czarnecki et al. 2017), wherein weight updates cannot be performed until the entire forward pass is completed. These constraints result in high memory overhead due to the need to store intermediate activations and hinder layer-wise parallelism, ultimately rendering BP inefficient for deployment on resource-constrained hardware.

To overcome the limitations of backpropagation, numerous alternatives have been proposed that either avoid or approximate its core mechanism. Feedback Alignment (FA) (Lillicrap et al. 2016) eliminates the requirement for weight symmetry in the backward pass by using fixed, randomly initialized feedback weights. DFA (Nøkland 2016) extends this by propagating output layer errors to hidden layers, bypassing the backward pathway altogether. Another related method, rLRA (Ororbia et al. 2023) enables weight updates without computing global gradients by recursively aligning each layer’s output with a synthetic target derived from the layer above. PEPITA (Dellafrera and Kreiman 2023)

replaces the backward pass with a second forward pass, in which the input is modulated according to the output-layer errors from the first forward pass. Notably, these approaches still necessitate full forward passes to compute global errors or store intermediate activations prior to updating weights. To eliminate update locking, local learning rules—which allow layers to update independently without waiting for the entire forward pass—are desirable. DRTP (Frenkel, Lefebvre, and Bol 2021) achieves this by directly projecting true labels to each layer, generating layer-wise error signals that support local updates without waiting for the full forward pass. Another approach is Hebbian learning (Hebb 2002), a biologically inspired local learning rule that strengthens synaptic connections when presynaptic and postsynaptic neurons are activated simultaneously. Despite its biological plausibility, Hebbian learning does not support supervised learning or error correction. SoftHebb (Moraitis et al. 2022; Journé et al. 2023) extends Hebbian learning by incorporating soft winner-take-all dynamics and differentiable surrogate gradients, enabling the training of deeper networks while preserving the characteristics of local learning. Similarly, Predictive Coding (Rao and Ballard 1999; Millidge, Seth, and Buckley 2022) offers a local learning framework in which each layer predicts the activity of the layer below, using the resulting prediction error to locally update both prediction and representation neurons.

The FF algorithm (Hinton 2022) introduces a fundamentally different training paradigm that entirely avoids the need for backpropagation. It employs a layer-wise learning mechanism in which each layer is independently supervised through two distinct forward passes—one with positive data and the other with negative data. Each layer computes a “goodness” score, defined as the sum of squared activations, and is trained using a binary cross-entropy loss to classify inputs as positive if the goodness exceeds a predefined threshold, and negative otherwise. Several extensions have been proposed to enhance the performance of FF. Contrastive Forward-Forward (Aghagolzadeh and Ezoji 2025) redefines goodness as the dot-product similarity between representations of different samples and introduces layer-wise supervised contrastive learning to improve the discriminative power of the network. SymBa (Lee and Song 2023) addresses the inherent asymmetry in binary cross-entropy loss by introducing a symmetric loss function with respect to positive and negative data, leading to improved convergence stability. SFFA (Terres-Escudero, Ser, and Bringas 2025) further enforces symmetry by dividing each layer’s neurons into positive and negative subsets, selectively activating them based on the input’s polarity. To overcome the lack of inter-layer interaction in FF, Layer Collaboration (Lorberbom et al. 2023) proposes a mechanism that allows goodness information to flow across layers, improving representational coherence and overall accuracy. However, these variants have not yet been extended to CNNs, limiting their applicability to more complex classification tasks.

Several recent efforts have explored adapting the FF paradigm to CNNs (Scodellaro et al. 2024; Chen et al. 2025) by injecting polarity information into the global image space to generate positive and negative samples instead of using

one-hot label embeddings as in the original FF. Other approaches take this further by eliminating the reliance on explicitly generated negative samples. For example, Cascaded Forward (CaFo) (Zhao et al. 2023) first trains the convolutional backbone using DFA, followed by training a set of layer-specific predictors, both stages relying solely on the original data. However, it suffers from slow convergence. A more recent state-of-the-art method is CwC learning (Papachristodoulou et al. 2024), which partitions the output channels of each layer based on the number of target classes. It calculates a group-wise goodness score for each class-aligned channel group and then uses these scores for classification (Figure 2). CwC has been shown to progressively enhance class-specific feature activation in deeper layers, partially addressing the representation degradation (Figure 1). DeeperForward (Sun et al. 2025) further extends CwC to deeper convolutional architectures and introduces a parallel training strategy to enhance efficiency. However, a significant limitation of CwC lies in its tight coupling between the number of channels and the number of classes. This design causes the channel dimensionality to scale linearly with the number of categories, severely limiting their performance on large-scale benchmarks and rendering them impractical for scaling to ImageNet. In light of these challenges, our work aims to further optimize FF-based training approaches for CNNs, with the goal of achieving improved scalability and performance on large-scale classification tasks.

Method

This section presents a backpropagation-free training framework built upon the proposed ASGE, a new mechanism that enhances representational capacity and overcomes the scalability limitations of prior FF-based methods in large-scale image classification tasks. ASGE trains each convolutional layer independently through channel-adaptive spatial goodness supervision, completely eliminating backward passes. We first introduce the core design of the ASGE mechanism, then detail the overall training framework.

Adaptive Spatial Goodness Encoding

Spatial Goodness Extraction Figure 2 illustrates the fundamental difference in goodness vector construction between our approach and CwC (Papachristodoulou et al. 2024). We propose a spatially grounded, class-agnostic method, where spatial goodness is defined as the energy within localized regions of each channel. This design explicitly decouples goodness computation from class count, enabling flexible supervision while ensuring scalability to large classification tasks. Formally, the feature maps of a given layer l are denoted as $\mathbf{Y}_l \in \mathbb{R}^{C_l \times H_l \times W_l}$ (post-activation), where C_l represents channels and $H_l \times W_l$ spatial dimensions. To standardize the quantification of localized regions, each $H_l \times W_l$ feature map is evenly partitioned into $P_l \times P_l$ patches. Accordingly, $\mathbf{Y}_l$ is reshaped into:

$$\tilde{\mathbf{Y}}_l \in \mathbb{R}^{C_l \times P_l \times P_l \times H'_l \times W'_l} \quad (1)$$

where $H'_l = H_l/P_l$, $W'_l = W_l/P_l$. The spatial goodness of each patch is computed as the mean squared activation

(energy) within the patch region:

$$g_{l,c}^{(i,j)} = \frac{1}{H'_l W'_l} \sum_{h=1}^{H'_l} \sum_{w=1}^{W'_l} \left(\tilde{\mathbf{Y}}_l^{(c,i,j,h,w)} \right)^2 \quad (2)$$

where $g_{l,c}^{(i,j)}$ denotes the spatial goodness of the (i,j) -th patch in channel c at layer l . The spatial goodness values of all patches across all channels are concatenated to form the goodness vector $\mathbf{g}_l \in \mathbb{R}^{1 \times (C_l \cdot P_l^2)}$.

Channel-Aware Patch Partitioning As discussed above, spatial goodness extraction relies on partitioning feature maps into patches. However, applying a fixed partitioning scheme uniformly across all layers overlooks the varying levels of feature abstraction captured at different depths of a CNN (Zeiler and Fergus 2013). To address this, we propose a channel-aware patch partitioning strategy, in which a partitioning factor P_l is adaptively scaled inversely with channel count C_l . Specifically, shallow layers tend to capture low-level, fine-grained features and thus benefit from finer spatial partitioning to produce sufficiently expressive goodness vectors for supervision. In contrast, deeper layers encode more abstract, high-level representations, where coarser spatial partitioning is sufficient to yield meaningful goodness vectors. This behavior is consistent with CNN architectures such as VGG (Simonyan and Zisserman 2015) and ResNet (He et al. 2015), where early layers typically have fewer channels and higher spatial resolutions, while deeper layers exhibit increased channel dimensionality and reduced spacial resolution. Consequently, it is intuitive to decrease P_l as C_l increases, as shown in Figure 2. We therefore compute P_l as:

$$P_l = \min \left(\max \left(1, \left\lfloor \frac{\alpha \cdot C_L}{C_l} \right\rfloor \right), H_l, W_l \right) \quad (3)$$

where C_L is the channel count of the final convolutional layer, and $\alpha \geq 0$ is a hyperparameter that controls partitioning granularity. This formulation ensures that $P_l \geq 1$, $H'_l \geq 1$ and $W'_l \geq 1$, while adaptively reducing the number of patches P_l as channel dimensionality C_l increases. A larger α results in finer partitioning with more patches, allowing the model to capture more detailed spatial goodness patterns. However, excessively large α may not yield further performance improvements. The effect of varying α will be further examined in the experimental section.

Random Projection for Supervision The spatial goodness vectors obtained as described above are used for layer-wise supervision via classification. A common approach is to employ auxiliary classifiers (Belilovsky, Eickenberg, and Oyallon 2019; Nøkland and Eidnes 2019); however, this introduces additional parameters and requires local backpropagation through the auxiliary branches, undermining the goal of achieving fully backpropagation-free training. To address this, we adopt a fixed random projection approach inspired by prior works (Huang, Zhu, and Siew 2006; Huang and Aminifar 2025; Apolinario, Roy, and Roy 2025). Specifically, each layer’s goodness vector is projected into the N -dimensional class space via a *linear* transformation defined

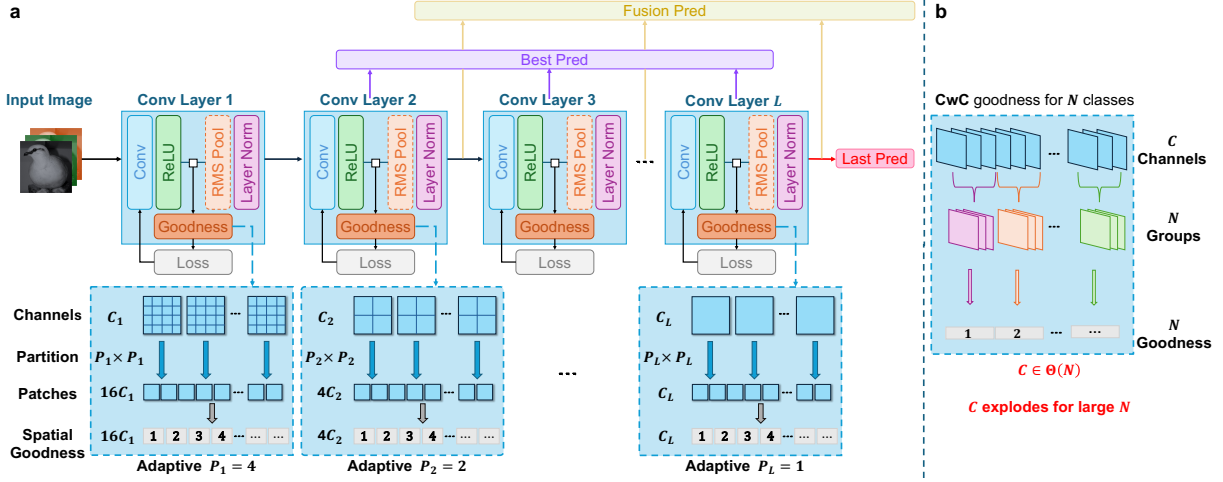


Figure 2: (a) Top: ASGE architecture with prediction strategies (*Fusion Pred*: Cumulative Layer Fusion Prediction, *Best Pred*: Best Layer Selection Prediction, *Last Pred*: Last Layer Only Prediction); Bottom: Layer-wise adaptive spatial goodness. (b) CwC goodness.

by a weight matrix $\mathbf{W}_l \in \mathbb{R}^{(C_l \cdot P_l^2) \times N}$ and a bias vector $\mathbf{b}_l \in \mathbb{R}^{1 \times N}$, both of which are randomly initialized at the beginning of training and remain fixed throughout. The generation procedure for these weight and bias parameters is as follows:

$$\mathbf{W}_l \sim \mathcal{N}\left(0, \frac{1}{\sqrt{N}}\right), \mathbf{b}_l \sim \mathcal{N}\left(0, \frac{1}{\sqrt{N}}\right) \quad (4)$$

The scaling factor $1/\sqrt{N}$ for $\mathbf{W}_l$ follows the Johnson-Lindenstrauss lemma, ensuring that the ℓ_2 norm of the projected vector remains close to that of the original goodness vector (Johnson and Lindenstrauss 1984). The bias vector $\mathbf{b}_l$ is similarly scaled by $1/\sqrt{N}$, serving as a static offset that improves the logit distribution after projection. The classification logits $\mathbf{a}_l \in \mathbb{R}^{1 \times N}$ are obtained from $\mathbf{g}_l$ via:

$$\mathbf{a}_l = \mathbf{g}_l \mathbf{W}_l + \mathbf{b}_l \quad (5)$$

which is then passed through a local cross-entropy loss $\mathcal{L}_l(\mathbf{a}_l; t^*)$, computed against the ground-truth target t^* :

$$\mathcal{L}_l(\mathbf{a}_l; t^*) = -\mathbf{a}_l^{(t^*)} + \log \sum_{n=1}^N \exp(\mathbf{a}_l^{(n)}) \quad (6)$$

The loss is subsequently used to compute gradients with respect to the convolutional parameters θ_l of layer l :

$$\nabla_{\theta_l} \mathcal{L}_l = \left(\frac{\partial \mathcal{L}_l}{\partial \mathbf{a}_l} \mathbf{W}_l^\top \right) \frac{\partial \mathbf{g}_l}{\partial \mathbf{Y}_l} \frac{\partial \mathbf{Y}_l}{\partial \theta_l} \quad (7)$$

This formulation allows the loss to directly produce gradients for the convolutional parameters, eliminating the need for local backpropagation through auxiliary trainable layers.

Training Framework

Overview As illustrated in Figure 2, during one forward pass, each layer is trained independently using spatial

goodness-based supervision. Specifically, the convolutional output at each layer is first passed through a ReLU nonlinearity. The resulting activated feature maps are then used to compute the layer’s spatial goodness and the corresponding loss, while simultaneously being forwarded to the subsequent operations—pooling (if present) and layer normalization (Ba, Kiros, and Hinton 2016). Critically, each layer’s output is detached before propagation to downstream layers, preventing gradient flow across layers and maintaining strict layer-wise independence.

Goodness-Preserving Pooling Pooling is a standard downsampling operation in CNNs, used to reduce spatial resolution and computational complexity. However, to preserve spatial goodness information during downsampling, the pooling method must align with the energy-based formulation used to compute goodness. Standard approaches like max pooling and average pooling distort energy representation: the former tends to exaggerate energy by focusing on extreme values, while the latter may suppress it by smoothing activations. To maintain compatibility with our energy-based goodness metric, we adopt an approach inspired by Learned-Norm Pooling (Gulcehre et al. 2014) which generalizes pooling through the use of L_p norm. Specifically, we set $p = 2$, resulting in a root mean square (RMS) pooling. Given a localized region $\mathcal{R} \in \mathbb{R}^{H \times W}$ of a feature map, RMS pooling is defined as:

$$\text{RMSPool}(\mathcal{R}) = \sqrt{\frac{1}{HW} \sum_h \sum_w (\mathcal{R}^{(h,w)})^2} \quad (8)$$

RMS pooling preserves the fundamental energy relationship, which directly aligns with our mean-squared-activation-based goodness computation, thus serves as an effective goodness-preserving downsampling mechanism.

Prediction Strategies We introduce three strategies (Figure 2) for generating the final prediction during training and

inference: (1) Last Layer Only prediction (*Last Pred*), (2) Cumulative Layer Fusion prediction (*Fusion Pred*), and (3) Best Layer Selection prediction (*Best Pred*).

In the *Last Pred* strategy, only the final convolutional layer’s features are passed through global average pooling (GAP), flattened, and fed into a single-layer linear classifier. During training, the resulting loss is used exclusively to update the classifier’s parameters, leaving earlier convolutional layers unaffected.

Given that each convolutional layer in our framework receives direct supervision from the ground-truth labels, each is expected to learn features that are independently useful for classification. To capitalize on this, the *Fusion Pred* strategy aggregates information from all convolutional layers except the first—excluded due to its low prediction accuracy. Specifically, we apply GAP and flattening to the feature maps of each selected layer, then concatenate the resulting representations. This combined vector is then passed to a single-layer linear classifier. Similar to the *Last Pred* strategy, only the classifier’s parameters are updated by the corresponding loss during training. This strategy effectively captures a broader range of features by leveraging the predictive capacity of multiple layers. However, it introduces additional classifier parameters and incurs memory overhead due to the need to store intermediate representations. This approach draws inspiration from the original FF method introduced in (Hinton 2022), but is specifically adapted to suit a convolutional architectures.

In the *Best Pred* strategy, each convolutional layer produces a classification output and associated prediction accuracy during training. The final prediction is then selected from the layer that achieves the highest validation accuracy. This approach offers two key advantages: (1) it eliminates the need for a final linear classifier, and (2) it enables early exit from inference when the most accurate prediction arises from a shallower layer, reducing computational cost.

Experiments

We evaluate our approach on five benchmark datasets: MNIST (Lecun et al. 1998), FashionMNIST (F-MNIST) (Xiao, Rasul, and Vollgraf 2017), CIFAR-10, CIFAR-100 (Krizhevsky and Hinton 2009), and ImageNet (ILSVRC 2012). To assess scalability, we include ImageNet’s 1.28 million training and 50,000 validation images across 1000 classes. For MNIST and F-MNIST, the original training sets are split into 50,000 training and 10,000 validation samples, and models are trained for 100 epochs. For CIFAR-10 and CIFAR-100, we use 45,000 training and 5,000 validation samples, with training conducted over 400 epochs. For all datasets except ImageNet, we adopt the AdamW optimizer with a weight decay of 0.001, a batch size of 128 and an initial learning rate of 0.0002, which is decayed to 0.00001 using a cosine annealing scheduler. For ImageNet, we use the SGD optimizer with momentum 0.9 and weight decay 0.0001. The batch size is 512, and the learning rate is scheduled from 0.05 to 0.0005 via cosine annealing. Models are trained for 45 epochs. Training is performed on 2 NVIDIA A100 GPUs (80 GB) and 1 Intel(R) Xeon(R) Gold 6338 CPU. We report the mean accuracy and standard deviation

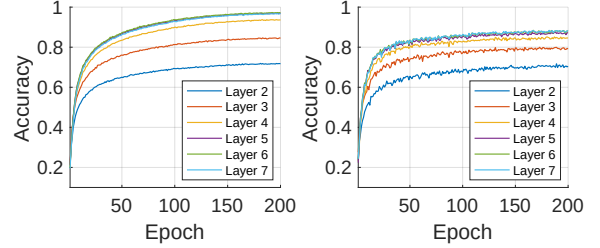


Figure 3: Layer-wise prediction accuracy (layers 2-7) of VGG8 trained with ASGE on CIFAR-10. First and final layers excluded. Left: training. Right: validation.

averaged over three independent runs. In each run, the model checkpoint with the highest validation accuracy is selected for final evaluation on the test set. Additional details, including model architectures, data augmentation strategies and so on, are provided in the supplementary materials.

Ablation Study

Effect of ASGE Figure 1 demonstrated a core limitation of FF training: a progressive degradation of layer-wise prediction accuracy with increasing network depth in CNNs. To assess whether ASGE mitigates this issue, Figure 3 presents the layer-wise accuracy of our VGG8 architecture, excluding the first layer—due to its negligible predictive performance—and the final linear classifier. The results reveal a fundamental reversal: from layers 2–7, prediction accuracy consistently improves across both the training and validation sets. Notably, the inter-layer accuracy gain diminishes with depth, indicating stabilization of increasingly refined feature representations. Overall, these findings confirm that ASGE successfully counteracts the FF’s representational degradation, enabling effective information accumulation across layers for scalable deep network learning.

Effect of Partitioning Factor α To examine the impact of the hyperparameter α on ASGE performance, we evaluate five representative values: 0, 0.5, 1, 1.5, and 2. All other training configurations are held constant, including the use of RMS pooling and the *Fusion Pred* strategy for final prediction. The results are summarized in Table 1. Across all datasets, a consistent trend emerges: test accuracy improves as α increases from 0 to 1, but begins to decline when α exceeds 1. The best performance is achieved at $\alpha = 1$, yielding test accuracies of 99.65% on MNIST, 93.41% on F-MNIST, 90.50% on CIFAR-10, and 65.34% on CIFAR-100. We interpret this behavior as follows: increasing α results in more spatial partitions per layer, allowing each layer to capture richer and more fine-grained spatial goodness information, which enhances performance. However, when α grows too large, the surplus patches introduces redundant information and noise, causing potential overfitting and degraded generalization. Thus, $\alpha = 1$ provides an effective balance between spatial granularity and noise, enabling optimal performance.

Effect of RMS Pooling Figure 4 visualizes the impact of different pooling operations on spatial goodness distributions. The first two columns show the distributions from the

ASGE (α)	Model	Partition (P_l)	MNIST	F-MNIST	CIFAR-10	CIFAR-100
0.0	VGG8	{1, 1, 1}	99.61 $\pm$ 0.01	92.88 $\pm$ 0.08	88.27 $\pm$ 0.20	61.69 $\pm$ 0.23
0.5	VGG8	{2, 1, 1}	99.64 $\pm$ 0.03	92.79 $\pm$ 0.10	89.20 $\pm$ 0.07	62.31 $\pm$ 0.29
1.0	VGG8	{4, 2, 1}	99.65 $\pm$ 0.03	93.41 $\pm$ 0.14	90.50 $\pm$ 0.08	65.34 $\pm$ 0.31
1.5	VGG8	{6, 3, 1}	99.63 $\pm$ 0.01	93.36 $\pm$ 0.07	90.22 $\pm$ 0.13	64.84 $\pm$ 0.35
2.0	VGG8	{8, 4, 2}	99.57 $\pm$ 0.04	93.12 $\pm$ 0.10	86.91 $\pm$ 0.22	60.40 $\pm$ 0.23

Table 1: Test accuracy (%) of ASGE under different values of α across four datasets. VGG8 contains layers with $C_l \in \{128, 256, 512\}$ and *Partition* row shows the corresponding P_l assigned to each C_l according to formula (3).

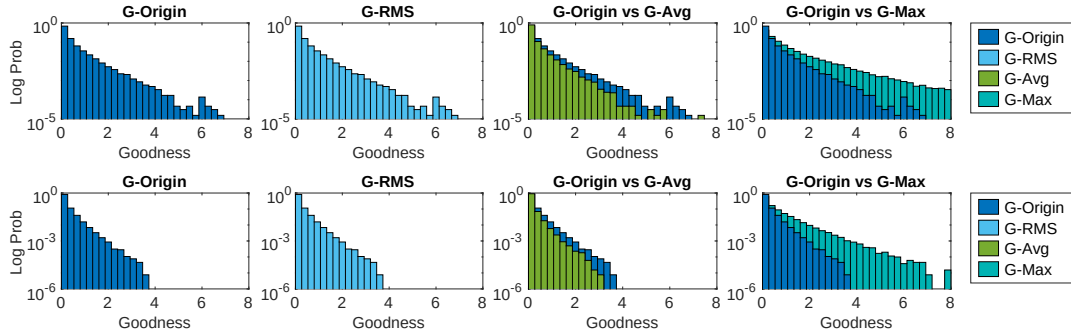


Figure 4: Spatial goodness distributions on a logarithmic scale before (G-Origin) and after different pooling (G-RMS, G-Avg, G-Max) in a layer that has pooling. Each row represents a different channel configuration: $C = 512$ (top) and $C = 256$ (bottom).

original representation (G-Origin) and after applying RMS pooling (G-RMS). The subsequent two columns compare G-Origin with the distributions obtained after average pooling (G-Avg) and max pooling (G-Max), respectively. It is evident that G-RMS closely mirrors G-Origin, preserving both the tail behavior and the overall distribution structure. In contrast, G-Avg distorts the original distribution by suppressing the tail and shifting toward lower goodness scores. G-Max, on the other hand, exaggerates the tail, increasing the proportion of higher goodness values. This indicates that RMS pooling more faithfully preserves spatial goodness information during downsampling, whereas the other two pooling methods introduce distributional distortions. Furthermore, we train a VGG8 using each pooling method with $\alpha = 1$ and the *Fusion Pred* strategy for final prediction. As shown in Table 2, ASGE with RMS pooling consistently outperforms average and max pooling across all datasets. By more efficiently preserving goodness information, RMS pooling aligns better with the core principles of ASGE and enhances the model’s representational capacity.

Performance of Prediction Strategies We evaluate the three proposed strategies for final prediction against standard BP using the same VGG8 with $\alpha = 1$ and RMS pooling, measuring test accuracy, classifier parameters, and intermediate activation memory. The only architectural modification occurs in *Best Pred*, which does not require a final classifier. Thus, its last layer is replaced with a copy of the previous convolutional layer. As shown in Table 3, *Fusion Pred* achieves the highest accuracy among the three (65.24%), coming closest to that of BP, but at the cost of the largest number of classifier parameters and the highest in-

termediate activation memory, though still requiring $120\times$ less memory than BP. In contrast, *Best Pred* sacrifices only 1.74% accuracy compared to *Fusion Pred*, while completely eliminating the final classifier and storing intermediate activations. Since the seventh layer yields the best prediction, inference can terminate after this layer, further reducing computational overhead. *Last Pred* uses the same number of classifier parameters as BP and consumes $600\times$ less activation memory, but delivers the lowest accuracy. *Last Pred* underperforms due to less discriminative raw activations under ASGE’s goodness supervision. In summary, the proposed strategies offer a flexible trade-off between accuracy, memory consumption, and computational cost, enabling efficient deployment across various resource-constrained scenarios.

Comparison with Other Work

We benchmark ASGE against standard BP and diverse BP-free approaches, including both FF-based and non-FF methods. Among non-FF methods, we include FA (Bartunov et al. 2018), Difference Target Propagation (DTP) (Ernoul et al. 2022; Bartunov et al. 2018), DFA (Nøkland 2016; Crafton et al. 2019), SoftHebb (Journé et al. 2023), rLRA (Ororbias et al. 2023) and PEPITA (Dellafrera and Kreiman 2023)). For FF-based methods, we evaluate the original FF (Hinton 2022), CaFo (Zhao et al. 2023), SymBa (Lee and Song 2023), CwC (Papachristodoulou et al. 2024) and DeepForward (Sun et al. 2025). All experiments for ASGE are conducted using VGG8 and VGG11 with $\alpha = 1$, RMS pooling, and *Fusion Pred* for final prediction.

Table 4 shows that ASGE consistently outperforms prior FF-based methods across MNIST, F-MNIST, CIFAR-10, and CIFAR-100, and surpasses most non-FF BP-free meth-

ASGE (Pooling)	Model	MNIST	F-MNIST	CIFAR-10	CIFAR-100
Average	VGG8	99.62 $\pm$ 0.03	93.18 $\pm$ 0.10	90.35 $\pm$ 0.15	64.96 $\pm$ 0.16
Max	VGG8	99.61 $\pm$ 0.03	93.07 $\pm$ 0.16	90.46 $\pm$ 0.18	64.41 $\pm$ 0.11
RMS	VGG8	99.65 $\pm$ 0.03	93.41 $\pm$ 0.14	90.50 $\pm$ 0.08	65.34 $\pm$ 0.31

Table 2: Test accuracy (%) of ASGE with different pooling operations.

Prediction Strategy	Model	Test Accuracy (%)	Activation Memory (MB)	Classifier Params ($\times 10^4$)	Best Layer
BP (repr.)	VGG8	69.11 $\pm$ 0.31	150	5.13	-
ASGE Fusion Pred	VGG8	65.34 $\pm$ 0.31	1.25	25.66	-
ASGE Last Pred	VGG8	63.31 $\pm$ 0.20	0.25	5.13	-
ASGE Best Pred	VGG8	63.60 $\pm$ 0.44	0	0	7

Table 3: CIFAR-100: ASGE prediction strategies vs. BP. *Activation Memory*: calculated using a batch of 128. *Best Layer*: the layer with the highest validation accuracy.

Method	Model	MNIST	F-MNIST	CIFAR-10	CIFAR-100	ImageNet	
						Top-1	Top-5
BP (repr.)	VGG8	99.64 $\pm$ 0.03	94.29 $\pm$ 0.10	91.68 $\pm$ 0.08	69.11 $\pm$ 0.31	46.16	72.02
FA	MLP8	98.74	-	58.03	-	6.92	17.46
DTP	VGG6	98.93 $\pm$ 0.04	90.91 $\pm$ 0.17	85.33 $\pm$ 0.32	-	1.66	5.44
DFA	CNN5	98.98 $\pm$ 0.05	-	73.10 $\pm$ 0.50	41.00 $\pm$ 0.30	6.20	-
SoftHebb	SoftHebb	99.35 $\pm$ 0.03	-	80.31 $\pm$ 0.14	56.00	27.30	-
rLRA	ResNet18	98.18	88.13	93.58	-	69.22	87.96
PEPITA	CNN2	98.29 $\pm$ 0.13	-	56.33 $\pm$ 1.35	27.56 $\pm$ 0.60	-	-
FF	MLP4	98.69	-	59.00	-	-	-
CaFo	CNN3	98.80	-	67.43	40.76	-	-
SymBa	MLP3	98.58	-	59.09	29.28	-	-
CwC	CNN4	99.42 $\pm$ 0.08	92.31 $\pm$ 0.32	78.11 $\pm$ 0.44	51.23	-	-
DeeperForward	ResNet18	99.63 $\pm$ 0.04	93.13 $\pm$ 0.13	88.72 $\pm$ 0.17	53.09 $\pm$ 0.79	-	-
ASGE (ours)	VGG8	99.65 $\pm$ 0.03	93.41 $\pm$ 0.14	90.50 $\pm$ 0.08	65.34 $\pm$ 0.31	23.04	43.08
ASGE (ours)	VGG11	99.59 $\pm$ 0.06	92.98 $\pm$ 0.24	90.62 $\pm$ 0.17	65.42 $\pm$ 0.16	26.21	47.49

Table 4: Test accuracy (%) comparison: ASGE vs. standard BP, non-FF and FF-based BP-free methods.

ods. The only exception is rLRA, which achieves a 2.96% higher accuracy on CIFAR-10. Notably, on the more challenging CIFAR-100 dataset, ASGE shows a significant advantage, exceeding all previous FF-based methods by at least 12.33%. Compared to BP, ASGE delivers slightly lower but still competitive results, with accuracy gaps of just 0.88% on F-MNIST, 1.18% on CIFAR-10, and 3.77% on CIFAR-100.

To our knowledge, ASGE is the first FF-based method to successfully scale to ImageNet, achieving Top-1 and Top-5 accuracies of 26.21% and 47.49%, setting a new benchmark for FF-based training. ASGE outperforms non-FF BP-free methods like FA, DTP, and DFA, though it trails SoftHebb, rLRA, and standard BP. However, rLRA requires a full forward pass before updates, increasing memory usage, while SoftHebb is limited to specific network architectures. ASGE circumvents these constraints with fully layer-wise training and flexible integration with convolutional architectures.

Conclusion

This work introduces ASGE, a novel FF-based BP-free training framework for CNNs that overcomes key limitations of prior FF methods through channel-aware spatial goodness supervision, enhancing both representational capacity and scalability to large-scale datasets. Experiments demonstrate that ASGE consistently outperforms all FF-based methods across 5 image classification benchmarks, and further narrows the performance gap with BP. Notably, it marks the first successful application of FF-based training to ImageNet, demonstrating its potential as a scalable and effective BP-free paradigm for modern CNNs.

Limitations Despite its strengths, ASGE has certain limitations. As a greedy, layer-wise approach, it generally requires longer training time to converge than end-to-end BP. Currently, ASGE is only applied to CNNs, and its extension to other architectures such as Transformers or RNNs

remains to be explored. Additionally, while ASGE performs competitively across various datasets, it still trails some non-FF BP-free methods on ImageNet, indicating room for future improvement.

References

- Aghagolzadeh, H.; and Ezoji, M. 2025. Contrastive Forward-Forward: A Training Algorithm of Vision Transformer. arXiv:2502.00571.
- Apolinario, M. P. E.; Roy, A.; and Roy, K. 2025. LLS: Local Learning Rule for Deep Neural Networks Inspired by Neural Activity Synchronization. In *2025 IEEE/CVF Winter Conference on Applications of Computer Vision (WACV)*, 7807–7816. IEEE.
- Ba, J. L.; Kiros, J. R.; and Hinton, G. E. 2016. Layer Normalization. arXiv:1607.06450.
- Bartunov, S.; Santoro, A.; Richards, B. A.; Marris, L.; Hinton, G. E.; and Lillicrap, T. 2018. Assessing the Scalability of Biologically-Motivated Deep Learning Algorithms and Architectures. arXiv:1807.04587.
- Belilovsky, E.; Eickenberg, M.; and Oyallon, E. 2019. Greedy Layerwise Learning Can Scale to ImageNet. arXiv:1812.11446.
- Bengio, Y. 2014. How Auto-Encoders Could Provide Credit Assignment in Deep Networks via Target Propagation. arXiv:1407.7906.
- Bottou, L. 2010. Large-Scale Machine Learning with Stochastic Gradient Descent. In Lechevallier, Y.; and Saporta, G., eds., *Proceedings of COMPSTAT'2010*, 177–186. Heidelberg: Physica-Verlag HD. ISBN 978-3-7908-2604-3.
- Chen, X.; Liu, D.; Laydevant, J.; and Grollier, J. 2025. Self-Contrastive Forward-Forward Algorithm. arXiv:2409.11593.
- Crafton, B.; Parihar, A.; Gebhardt, E.; and Raychowdhury, A. 2019. Direct Feedback Alignment with Sparse Connections for Local Learning. arXiv:1903.02083.
- Czarnecki, W. M.; Świrszcz, G.; Jaderberg, M.; Osindero, S.; Vinyals, O.; and Kavukcuoglu, K. 2017. Understanding Synthetic Gradients and Decoupled Neural Interfaces. arXiv:1703.00522.
- Dellafrerra, G.; and Kreiman, G. 2023. Error-driven Input Modulation: Solving the Credit Assignment Problem without a Backward Pass. arXiv:2201.11665.
- Ernout, M.; Normandin, F.; Moudgil, A.; Spinney, S.; Belilovsky, E.; Rish, I.; Richards, B.; and Bengio, Y. 2022. Towards Scaling Difference Target Propagation by Learning Backprop Targets. arXiv:2201.13415.
- Frenkel, C.; Lefebvre, M.; and Bol, D. 2021. Learning Without Feedback: Fixed Random Learning Signals Allow for Feedforward Training of Deep Neural Networks. *Frontiers in Neuroscience*, 15.
- Goodfellow, I. J.; Pouget-Abadie, J.; Mirza, M.; Xu, B.; Warde-Farley, D.; Ozair, S.; Courville, A.; and Bengio, Y. 2014. Generative Adversarial Networks. arXiv:1406.2661.
- Grossberg, S. 1987. Competitive learning: From interactive activation to adaptive resonance. *Cognitive Science*, 11(1): 23–63.
- Gulcehre, C.; Cho, K.; Pascanu, R.; and Bengio, Y. 2014. Learned-Norm Pooling for Deep Feedforward and Recurrent Neural Networks. arXiv:1311.1780.
- He, K.; Zhang, X.; Ren, S.; and Sun, J. 2015. Deep Residual Learning for Image Recognition. arXiv:1512.03385.
- Hebb, D. O. 2002. *The organization of behavior : a neuropsychological theory*. Mahwah, N.J: L. Erlbaum Associates, 1st ed. edition. ISBN 1-135-63191-3.
- Hinton, G. 2022. The Forward-Forward Algorithm: Some Preliminary Investigations. arXiv:2212.13345.
- Huang, B.; and Aminifar, A. 2025. TinyFoA: Memory Efficient Forward-Only Algorithm for On-Device Learning. *Proceedings of the AAAI Conference on Artificial Intelligence*, 39(16): 17377–17385.
- Huang, G.-B.; Zhu, Q.-Y.; and Siew, C.-K. 2006. Extreme learning machine: Theory and applications. *Neurocomputing*, 70(1): 489–501.
- Johnson, W.; and Lindenstrauss, J. 1984. Extensions of Lipschitz maps into a Hilbert space. *Contemporary Mathematics*, 26: 189–206.
- Journé, A.; Rodriguez, H. G.; Guo, Q.; and Moraitis, T. 2023. Hebbian Deep Learning Without Feedback. arXiv:2209.11883.
- Kingma, D. P.; and Ba, J. 2017. Adam: A Method for Stochastic Optimization. arXiv:1412.6980.
- Krizhevsky, A.; and Hinton, G. 2009. Learning multiple layers of features from tiny images. Technical Report 0, University of Toronto, Toronto, Ontario.
- LeCun, Y.; Bengio, Y.; and Hinton, G. 2015. Deep learning. *Nature*, 521(7553): 436–444.
- Lecun, Y.; Bottou, L.; Bengio, Y.; and Haffner, P. 1998. Gradient-based learning applied to document recognition. *Proceedings of the IEEE*, 86(11): 2278–2324.
- Lee, H.-C.; and Song, J. 2023. SymBa: Symmetric Backpropagation-Free Contrastive Learning with Forward-Forward Algorithm for Optimizing Convergence. arXiv:2303.08418.
- Lillicrap, T. P.; Cownden, D.; Tweed, D. B.; and Akerman, C. J. 2016. Random synaptic feedback weights support error backpropagation for deep learning. *Nature Communications*, 7(1): 13276. Publisher: Nature Publishing Group.
- Lorberbom, G.; Gat, I.; Adi, Y.; Schwing, A.; and Hazan, T. 2023. Layer Collaboration in the Forward-Forward Algorithm. arXiv:2305.12393.
- Millidge, B.; Seth, A.; and Buckley, C. L. 2022. Predictive Coding: a Theoretical and Experimental Review. arXiv:2107.12979.
- Moraitis, T.; Toichkin, D.; Journé, A.; Chua, Y.; and Guo, Q. 2022. SoftHebb: Bayesian inference in unsupervised Hebbian soft winner-take-all networks. *Neuromorphic Computing and Engineering*, 2(4): 044017.

Nøkland, A. 2016. Direct Feedback Alignment Provides Learning in Deep Neural Networks. arXiv:1609.01596.

Nøkland, A.; and Eidnes, L. H. 2019. Training Neural Networks with Local Error Signals. arXiv:1901.06656.

Ororbia, A. G.; Mali, A.; Kifer, D.; and Giles, C. L. 2023. Backpropagation-Free Deep Learning with Recursive Local Representation Alignment. *Proceedings of the AAAI Conference on Artificial Intelligence*, 37(8): 9327–9335. Number: 8.

Papachristodoulou, A.; Kyrkou, C.; Timotheou, S.; and Theodoridis, T. 2024. Convolutional Channel-Wise Competitive Learning for the Forward-Forward Algorithm. *Proceedings of the AAAI Conference on Artificial Intelligence*, 38(13): 14536–14544.

Rao, R. P. N.; and Ballard, D. H. 1999. Predictive coding in the visual cortex: a functional interpretation of some extra-classical receptive-field effects. *Nature Neuroscience*, 2(1): 79–87.

Rumelhart, D. E.; Hinton, G. E.; and Williams, R. J. 1986. Learning representations by back-propagating errors. *Nature*, 323(6088): 533–536.

Russakovsky, O.; Deng, J.; Su, H.; Krause, J.; Satheesh, S.; Ma, S.; Huang, Z.; Karpathy, A.; Khosla, A.; Bernstein, M.; Berg, A. C.; and Fei-Fei, L. 2015. ImageNet Large Scale Visual Recognition Challenge. arXiv:1409.0575.

Scodellaro, R.; Kulkarni, A.; Alves, F.; and Schröter, M. 2024. Training Convolutional Neural Networks with the Forward-Forward algorithm. arXiv:2312.14924.

Simonyan, K.; and Zisserman, A. 2015. Very Deep Convolutional Networks for Large-Scale Image Recognition. arXiv:1409.1556.

Sun, L.; Zhang, Y.; He, W.; Wen, J.; Shen, L.; and Xie, W. 2025. DeeperForward: Enhanced Forward-Forward Training for Deeper and Better Performance. In *The Thirteenth International Conference on Learning Representations*.

Terres-Escudero, E. B.; Ser, J. D.; and Bringas, P. G. 2025. A Contrastive Symmetric Forward-Forward Algorithm (SFFA) for Continual Learning Tasks. arXiv:2409.07387.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2023. Attention Is All You Need. arXiv:1706.03762.

Xiao, H.; Rasul, K.; and Vollgraf, R. 2017. Fashion-MNIST: a Novel Image Dataset for Benchmarking Machine Learning Algorithms. arXiv:1708.07747.

Zeiler, M. D.; and Fergus, R. 2013. Visualizing and Understanding Convolutional Networks. arXiv:1311.2901.

Zhao, G.; Wang, T.; Li, Y.; Jin, Y.; Lang, C.; and Ling, H. 2023. The Cascaded Forward Algorithm for Neural Network Training. arXiv:2303.09728.